



UNIDADE II

Programação
Estruturada em C

Prof. Me. Fábio Assis

Estrutura Condicional em C

- A estrutura `if` permite que o programa execute ou ignore determinados blocos de código, dependendo de uma condição lógica avaliada como verdadeira (1) ou falsa (0).

Sintaxe:

```
if (condição) {  
    // Bloco de código executado se a condição for verdadeira  
}
```

- Se a condição for verdadeira (V), o programa executa as instruções dentro do bloco do `if`.
- Se a condição for falsa (F), o fluxo desvia para outro caminho.
- O programa continua após a execução ou não das instruções.

Estrutura Condicional em C

- A condição pode envolver operações (`==`, `!=`, `>`, `<` etc.) e operações lógicas (`&&`, `||`, `!`).
- O bloco de instruções é o conjunto de comandos que será executado apenas se a condição for atendida. Caso contrário, o programa ignora esse bloco e segue para a próxima instrução.

Exemplo:

```
int idade = 18;

if (idade >= 18) {
    printf("Pode votar.");
}
```

- A condição `idade >= 18` é avaliada. Como `idade` vale 18, o programa imprime "Pode votar."

Estrutura Condicional em C

Estrutura *if – else* permite ao programa escolher entre dois caminhos de execução: um quando a condição for verdadeira *if* e outro quando for falsa *else*.

Sintaxe:

```
if (condição) {  
    // Código executado se a condição for verdadeira  
}  
else {  
    // Código executado se a condição for falsa  
}
```

Estrutura	Funcionamento
<i>if</i>	Executa um bloco de código se a condição for verdadeira
<i>else</i>	Executa um bloco alternativo caso a condição do <i>if</i> seja falsa

Estrutura Condicional em C

Exemplo:

```
int idade = 20;
if (idade >= 18 && idade <= 60) {
    printf("Idade válida para cadastro");
}
else {
    printf("Idade fora do intervalo");
}
```

- Como idade é igual a 20, a condição *idade* >= 18 && *idade* ≤ 60 é avaliada como verdadeira (1). Portanto, dentro do intervalo válido.
- Logo, o programa imprime “Idade válida para cadastro”.

Estrutura Condicional Encadeada em C

- Em C, podemos usar estruturas condicionais encadeadas para testar múltiplas condições sequenciais. Isso é feito com o uso de *if*, *else if* e *else*.
- Exemplo: programa que permite imprimir uma mensagem dependendo do valor digitado pelo usuário.

```
#include <stdio.h>

int main() {
    int numero;

    printf("Digite um número: ");
    scanf("%d", &numero);

    if(numero > 0){
        printf("Número positivo.\n");
    }
    else if(numero == 0){
        printf("Número igual a zero.\n");
    }
    else{
        printf("Número negativo.\n");
    }

    return 0;
}
```

Interatividade

Analise o código escrito em C e marque a alternativa que apresenta a mensagem de saída na impressão deste programa. Considere que o usuário digitou o valor 30.

- a) Criança.
- b) Adolescente.
- c) Adulto.
- d) Idoso.
- e) Todas as mensagens.

```
#include <stdio.h>
int main() {
    int idade;

    printf("Digite a idade: ");
    scanf("%d", &idade);

    if(idade < 12){
        printf("Criança\n");
    }
    else if(idade < 18){
        printf("Adolescente\n");
    }
    else if(idade < 60){
        printf("Adulto\n");
    }
    else{
        printf("Idoso\n");
    }
    return 0;
}
```

Resposta

Analise o código escrito em C e marque a alternativa que apresenta a mensagem de saída na impressão deste programa. Considere que o usuário digitou o valor 30.

- a) Criança.
- b) Adolescente.
- c) Adulto.**
- d) Idoso.
- e) Todas as mensagens.

```
#include <stdio.h>
int main() {
    int idade;

    printf("Digite a idade: ");
    scanf("%d", &idade);

    if(idade < 12){
        printf("Criança\n");
    }
    else if(idade < 18){
        printf("Adolescente\n");
    }
    else if(idade < 60){
        printf("Adulto\n");
    }
    else{
        printf("Idoso\n");
    }
    return 0;
}
```


Estruturas de Repetição em C

Estruturas de repetição, também conhecidas como laços de repetição ou loops, são essenciais em programação e você as utilizará com muita frequência ao iniciar seus estudos em qualquer linguagem. Em C, existem três estruturas principais de repetição:

- *while*
- *do – while*
- *for*
- Um loop é um trecho de código que será repetido enquanto uma condição lógica for verdadeira. Assim que essa condição deixar de ser satisfeita, o programa interrompe a repetição e continua a execução normalmente.

Tipos de repetição: Existem basicamente dois tipos principais:

- Repetição determinada: controlada por um contador.
- Repetição indeterminada: controlada por uma sentinela.

Estruturas de Repetição: while

- Na linguagem C, o laço *while* é uma estrutura de repetição usada para executar um bloco de código enquanto uma condição for verdadeira.
- A condição é verificada antes de cada repetição; se ela for falsa desde o início, o bloco não será executado nenhuma vez.

Sintaxe:

```
// Inicialização da variável de controle
while (condição) {
    // Bloco de código a ser executado
    // Atualização da variável de controle
}
```

- Inicialização da variável de controle (antes do *while*).
- Bloco de código a ser executado enquanto a condição for verdadeira.
- Atualização da variável de controle (para evitar loop infinito).

Estruturas de Repetição: while

- Exemplo: Programa para imprimir uma sequência numérica predefinida.

Saída na tela:

```
1
2
3
4
5
```

```
#include <stdio.h>

int main() {
    int i = 1; // inicialização

    while (i <= 5) { // condição
        printf("%d\n", i); // impressão na tela
        i++; // atualização
    }

    return 0;
}
```

Estruturas de Repetição: while

- Exemplo: No exemplo a seguir, o programa solicita que o usuário insira notas continuamente. A entrada termina apenas quando o valor -1 for digitado, servindo como valor de parada. Uso de sentinela (repetição indeterminada).

```
#include <stdio.h>

int main() {
    int nota;
    printf("Digite uma nota (-1 para encerrar): ");
    scanf("%d", &nota);

    while (nota != -1) {
        printf("Nota registrada: %d\n", nota);
        printf("Digite uma nota (-1 para encerrar): ");
        scanf("%d", &nota);
    }

    return 0;
}
```

Estruturas de Repetição: do-while

- O loop *do – while* é uma estrutura de repetição usada em C quando precisamos garantir que uma ação aconteça pelo menos uma vez, mesmo que a condição seja falsa logo no início.

Sintaxe:

```
do {  
    // código que será repetido  
} while (condição);
```

O que acontece?

- O bloco de código é executado primeiro. Depois, a condição é verificada.
- Se a condição for verdadeira, o laço repete.
- Se a condição for falsa, o laço termina.

Interatividade

Analise o código escrito em C e as afirmações:

- I. O menu será mostrado ao menos uma vez.
- II. O programa aguarda o usuário digitar um valor.
- III. O loop continua enquanto `opcao != 0`.

Está correto o que se afirma em:

- a) I e II, apenas.
- b) I e III, apenas.
- c) I, apenas.
- d) I, II e III.
- e) II, apenas.

```
#include <stdio.h>

int main() {

    int opcao;

    do {
        printf("MENU PRINCIPAL\n");
        printf("1 - Cadastrar\n");
        printf("2 - Consultar\n");
        printf("0 - Sair\n");
        printf("Escolha uma opção: ");
        scanf("%d", &opcao);
    } while (opcao != 0);

    return 0;
}
```

Resposta

Analise o código escrito em C e as afirmações:

- I. O menu será mostrado ao menos uma vez.
- II. O programa aguarda o usuário digitar um valor.
- III. O loop continua enquanto `opcao != 0`.

Está correto o que se afirma em:

- a) I e II, apenas.
- b) I e III, apenas.
- c) I, apenas.
- d) I, II e III.
- e) II, apenas.

```
#include <stdio.h>

int main() {

    int opcao;

    do {
        printf("MENU PRINCIPAL\n");
        printf("1 - Cadastrar\n");
        printf("2 - Consultar\n");
        printf("0 - Sair\n");
        printf("Escolha uma opção: ");
        scanf("%d", &opcao);
    } while (opcao != 0);

    return 0;
}
```

Estruturas de Repetição: for

- O laço (loop) *for* é uma estrutura de repetição muito usada na programação, especialmente quando sabemos exatamente quantas vezes queremos que uma ação aconteça.

Sintaxe: **for**(inicialização; condição; atualização) {
 // bloco de código a ser repetido
}

- Inicialização: define uma variável de controle (geralmente chamada de contador) e executa essa etapa apenas uma vez, no início do loop.
 - Condição: é verificada antes de cada repetição (ou iteração). Se for verdadeira, o bloco de código dentro do loop é executado. Se for falsa, o loop é encerrado.
 - Atualização: modifica o valor do contador ao final de cada iteração. É normalmente usada para avançar para o próximo passo.

Estruturas de Repetição: for

Exemplo: Enviar e-mails automáticos para 100 clientes:

```
#include <stdio.h>

int main() {
    int i; //declaração da variável de controle

    for(i = 1; i <= 100; i++){
        printf("Enviando e-mail para o cliente %d\n", i);
    }

    return 0;
}
```

- Neste exemplo será impressa a mensagem "Enviando e-mail para o cliente %d\n", 100 vezes, uma em cada linha.

Estruturas de Repetição: for

- Exemplo: Programa para somar as notas de 10 alunos e calcular a média da turma.

```
#include <stdio.h>

int main() {
    int i;
    float nota, soma = 0, media;

    for (i = 1; i <= 10; i++) {
        printf("Digite a nota do aluno %d: ", i);
        scanf("%f", &nota);
        soma += nota; // soma = soma + nota
    }

    media = soma / 10;
    printf("Média da turma: %.2f\n", media);

    return 0;
}
```

- Neste exemplo será impressa a média da turma, exibida com duas casas decimais (%.2f).

Interatividade

Analise o código escrito em C que imprime os números inteiros ímpares de 1 até n . Considere que foi digitado o valor 10.

Assinale a alternativa correta:

- a) 1 2 3 4 5 6 7 8 9 10.
- b) 0 2 4 6 8 10.
- c) 1 3 5 7 9.
- d) 9 7 5 3 1.
- e) 10 9 8 7 6 5 4 3 2 1.

```
#include <stdio.h>

int main() {
    int n, i;

    printf("Digite um número: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++) {
        if (i % 2 == 1) {
            printf("%d ", i);
        }
    }

    return 0;
}
```

Resposta

Analise o código escrito em C que imprime os números inteiros ímpares de 1 até n . Considere que foi digitado o valor 10.

Assinale a alternativa correta:

- a) 1 2 3 4 5 6 7 8 9 10.
- b) 0 2 4 6 8 10.
- c) 1 3 5 7 9.
- d) 9 7 5 3 1.
- e) 10 9 8 7 6 5 4 3 2 1.

```
#include <stdio.h>

int main() {
    int n, i;

    printf("Digite um número: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++) {
        if (i % 2 == 1) {
            printf("%d ", i);
        }
    }

    return 0;
}
```

Estruturas de Repetição – Aninhado

- Laços aninhados (encadeado), ou seja, um laço dentro de outro: são úteis em sistemas que geram relatórios, cálculos repetitivos ou comparações cruzadas. Além disso, os laços aninhados são essenciais para trabalharmos com matrizes.

Exemplos de programas com laços aninhados:

- Geração de tabuadas de 1 até 10, todas de uma vez.
- Triângulo de asteriscos com n linhas.
- Jogos com matrizes como jogo da velha.
- Esses exemplos de códigos podem ser feitos com (*for*, *while* ou *do – while*).

Estruturas de Repetição – Aninhado: for

- Exemplo: programa em C para imprimir um triângulo de números de 1 até n .

Exemplo com $n = 3$:

A execução ficaria assim:

i = 1: imprime:	1
i = 2: imprime:	1 2
i = 3: imprime:	1 2 3

```
#include <stdio.h>
```

```
int main() {
```

```
    int n=0, i=0, j=0;
```

```
    printf("Digite o número de linhas: ");
```

```
    scanf("%d", &n);
```

```
    for(i = 1; i <= n; i++) {
```

```
        for (j = 1; j <= i; j++) {
```

```
            printf("%d ", j);
```

```
        }
```

```
        printf("\n");
```

```
    }
```

```
    return 0;
```

```
}
```

Estrutura Switch-case

- A estrutura switch é uma forma de seleção múltipla para permitir que o programa execute diferentes blocos de código, de acordo com o valor de uma variável.
- O switch pode ser utilizado como alternativa ao uso de múltiplas instruções if-else if.
- A instrução switch avalia o valor de uma expressão e o compara com os valores definidos em cada case. Quando ocorre uma correspondência, o bloco de código relacionado é executado.

Estrutura básica:

```
switch (expressao) {  
    case constante 1:  
        //Bloco executado se expressao igual constante1  
        break; //interrompe imediatamente a execução  
    case constante 2:  
        //Bloco executado se expressao igual constante2  
        break;  
    case constante n:  
        //Bloco executado se expressao igual constanteN  
        break;  
    default:  
        //Bloco executado se não entrar em nenhum case  
}  
}
```

Interatividade

Dado o valor 5 fornecido pelo usuário, examine o trecho de código em linguagem C e escolha a resposta que corresponde corretamente à saída do programa:

- a) Você atacou o inimigo!
- b) Você defendeu o ataque!
- c) Você desistiu da batalha.
- d) Você venceu a batalha.
- e) Opção inválida.

```
int opcao;  
scanf("%d", &opcao);  
  
switch(opcao) {  
    case 1:  
        printf("Você atacou o inimigo!\n");  
        break;  
    case 2:  
        printf("Você defendeu o ataque!\n");  
        break;  
    case 3:  
        printf("Você desistiu da batalha.\n");  
        break;  
    default:  
        printf("Opção inválida.\n");  
}
```


Resposta

Dado o valor 5 fornecido pelo usuário, examine o trecho de código em linguagem C e escolha a resposta que corresponde corretamente à saída do programa:

- a) Você atacou o inimigo!
- b) Você defendeu o ataque!
- c) Você desistiu da batalha.
- d) Você venceu a batalha.
- e) **Opção inválida.**

```
int opcao;  
scanf("%d", &opcao);  
  
switch(opcao) {  
    case 1:  
        printf("Você atacou o inimigo!\n");  
        break;  
    case 2:  
        printf("Você defendeu o ataque!\n");  
        break;  
    case 3:  
        printf("Você desistiu da batalha.\n");  
        break;  
    default:  
        printf("Opção inválida.\n");  
}
```

Referências

- BACKES, André. *Linguagem C: completa e descomplicada*. São Paulo: Grupo GEN, 2018.
- CELES, Waldemar; RANGEL, Renato; EIRAS, José. *Introdução a estruturas de dados: com técnicas de programação em C*. Rio de Janeiro: Elsevier/Grupo GEN, 2016.
- DAMAS, Luiz. *Linguagem C*. Rio de Janeiro: LTC, 2011.
- DEITEL, Paul; DEITEL, Harvey. *C: como programar*. São Paulo: Pearson Prentice Hall, 2013.
- MANZANO, José Augusto N. G.; OLIVEIRA, Jayr Figueiredo de. *Algoritmos: lógica para desenvolvimento de programação de computadores*. São Paulo: Saraiva, 2016.

ATÉ A PRÓXIMA!